

“OSI Layer Crash Course: Outage Edition”

A clean, beginner-friendly, outage-focused version that builds the foundational thinking patterns needed for real war rooms.

This is NOT the BattleOps deep OSI analysis (multiple layers, packet tracing, TCP state diagrams).

This is the **CoreOps edition**: simple, practical, precise, written for engineers who are beginning their war-room journey.

Why OSI Layer Thinking Matters in Production Debugging

Most outages in Kubernetes **look complicated**, but actually map to one of the 7 OSI layers.

If you THINK in layers, you **never get confused**.

Layer thinking gives you:

- A structured approach
- A mental model to avoid chaos
- A way to isolate failures
- The ability to predict what is *not* the problem

CoreOps goal: Learn to *quickly* map symptoms → OSI layers → correct troubleshooting commands.

1. The InfraThrone OSI View for CoreOps

We don't memorize OSI 1–7.

We use the **InfraThrone collapse model** → simplify 7 layers into **4 practical buckets**.

These buckets align with Kubernetes debugging.

BUCKET 1: Physical Layer (Nodes, CPU, Memory, Disk, Kubelet)

“Is the machine itself healthy?”

In the Kubernetes world, this means:

- Node is Ready / NotReady
- Disk pressure
- Memory pressure
- CPU starvation
- Kubelet failing
- Container runtime stuck



Symptoms that map here:

- Pods stuck in **ContainerCreating**
- Node shows **NotReady**
- Frequent OOMKills across pods
- Kubelet unable to pull images
- DiskFull → container cannot write logs

Commands (CoreOps Edition)

1. `kubectl get nodes`
2. `kubectl describe node <node>`
3. `kubectl top nodes`
4. `kubectl top pod -A`
5. `kubectl get events --sort-by=.metadata.creationTimestamp`

Quick isolation question: “If the node is sick, can any pod on that node be healthy?”

→ **No.**

BUCKET 2: Network Layer (Service, Endpoints, DNS, Connectivity)

“Is the traffic flowing?”

This is where **MOST CoreOps outages happen.**

Kubernetes networking =

- DNS
- CNI
- Service → Endpoints
- Pod-to-pod connectivity

Symptoms that map here:

- Service reachable from one pod but not another
- 503 upstream unavailable
- `curl <service>:<port>` returns timeout
- Endpoints: `<none>`
- DNS resolution failures

- Pods cannot talk across namespaces

Commands

1. `kubectl get svc -A`
2. `kubectl get ep -A`
3. `kubectl exec -it <pod> -- curl <service>:<port>`
4. `kubectl exec -it <pod> -- nslookup <service>`
5. `kubectl get networkpolicy -A`

Quick isolation question: “Does the service have endpoints?”

→ If NO, debugging shifts to the pod lifecycle.

BUCKET 3: Transport Layer (Ports, Probes, Timeouts, TLS)

“Is the application reachable on the correct port and behaving?”

This layer includes:

- Liveness/readiness probe failures
- Port mismatch
- Container not listening
- Timeout issues
- TLS handshake failures

Symptoms that map here:

- Pod is running, but **Ready = false**
- Liveness probe failed
- Ports wrong in Deployment vs Service
- App not listening inside the container
- TLS handshake timeout

Commands

1. `kubectl describe pod <name> | grep -i probe`
2. `kubectl exec -it <pod> -- netstat -tulnp`
3. `kubectl exec -it <pod> -- curl localhost:<port>`
4. `kubectl get deploy <name> -o yaml | grep -i port`
5. `kubectl logs <name>`

Quick isolation question: “Does the app inside the pod respond locally?”

→ **If NO, it's the app container itself.**

BUCKET 4: Application Layer (Config, Env Vars, Code, Dependencies)

“Everything looks healthy, but the app is misbehaving.”

This includes:

- Wrong env vars
- Missing configmaps
- Wrong image tag deployed
- Code errors
- Dependency failures (DB, Redis, S3, APIs)
- Rate limits

Symptoms that map here:

- 500 errors
- App looks Running but throws errors
- ConfigMap changes not picked up
- API downstream failing
- DB connection refused

Commands

1. `kubectl logs <pod>`
2. `kubectl describe deploy <name> | grep -i image`
3. `kubectl get configmap <name> -o yaml`
4. `kubectl exec -it <pod> -- env`

Quick isolation question: “Is the pod healthy, but the API still failing?”

→ **It’s almost always the application layer.**

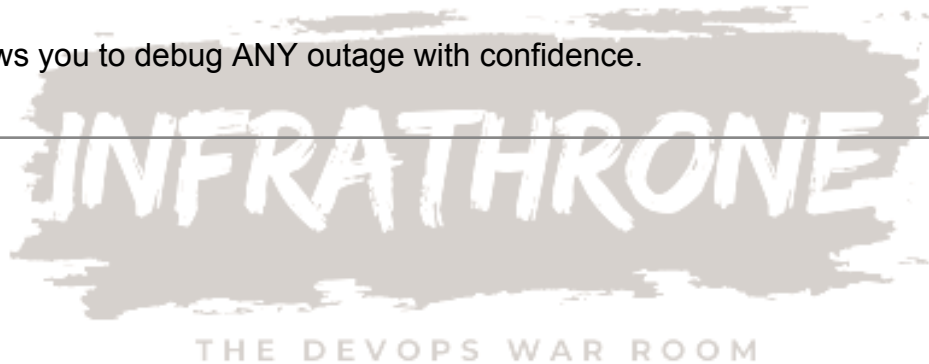
2. CoreOps Flow: How OSI Helps in Real Outages

A CoreOps engineer must answer this quickly:

“Which OSI bucket does the failure belong to?”

“What are the first 3 commands for that bucket?”

This allows you to debug ANY outage with confidence.



3. OSI → Command Mapping Table (CoreOps Cheatsheet)

OSI Bucket	Example Symptoms	First 3 Commands
Node / Physical	Pod Pending, Node NotReady, OOMKilled	get nodes, describe node, top nodes
Network	Service unreachable, DNS fail	get ep, curl, nslookup
Transport	Probes failing, port mismatch	Describe pod, curl localhost, netstat
Application	500 errors, wrong env vars	logs, get deploy -o yaml, env

THE DEVOPS WAR ROOM

4. How a CoreOps Engineer Should Use This in Real Life

FIRST RULE: Never jump to random commands.

SECOND RULE: Always identify the OSI bucket first.

THIRD RULE: Only run commands from the bucket you're investigating.

This reduces:

- Noise
- Time wasted
- Wrong assumptions
- Panic

And slowly builds true production intuition.

5. CoreOps Practice: 4 Mini Labs to Train OSI Thinking

These 4 labs will make beginners THINK like L2 engineers:

Lab 1: Service has no endpoints

Category → **Network layer**

Goal → Understand svc <→ pod relationship.

Lab 2: Liveness probe failing

Category → **Transport layer**

Goal → Understand how probes control availability.

Lab 3: Node memory pressure

Category → **Physical layer**

Goal → Correlate pod evictions + node-level issues.

Lab 4: Wrong env var breaks app

Category → **Application layer**

Goal → Understand how small config mistakes create outages.

6. Why OSI Helps Even in Kubernetes

People think OSI is outdated.

But in production debugging, OSI = the only way to:

- Categorize issues
 - Avoid emotional debugging
 - Train hypothesis-based thinking
 - Move towards war-room level skills
-

7. Final InfraThrone Message for CoreOps

This crash course is your stepping stone into advanced debugging.

Master this → You enter BattleOps with **clarity** instead of confusion.

You will never again ask: “Where should I start troubleshooting?”

Instead, you’ll say: “**Based on the symptom, this belongs to Bucket 2 → Network. Let’s run these three commands first.**” This is how CoreOps evolves into BattleOps.

THE DEVOPS WAR ROOM